MONDE

Méthode d'actualisation **2**

Entités **6**

Le monde **7**

Ajouter une entitée 8

Trouver une entitée 10

Enlever une entitée 11

Itérateurs **12**

Manipulation de donnée dans une boucle 12

Scènes **14**

La partie théorique de l'examen s'appuie quasiment entièrement sur les chapitres à lire dans le livre « *Game Programming Patterns* » de Robert Nystrom.

Certains sujets ne seront **pas directement abordés** dans le cours et c'est à vous de lire les chapitres et de poser des questions au besoin.

Lecture :

Robert Nystrom, Game Programming Patterns, Update Method
https://gameprogrammingpatterns.com/update-method.html

Méthode d'actualisation

À chaque mise à jour, la boucle principale d'un jeu appelle du code pour actualiser la logique des éléments qu'il contient. Que ce soit pour les déplacements d'un personnage, les calculs d'un IA ou la simulation de la physique, un programme en temps réel continue de se rafraîchir sans arrêt.

Prenons par exemple, le code suivant :

```
class Goblin {
public:
    void SetX(float x) { m_X = x; }
    void SetY(float y) { m_Y = y; }

private:
    float m_X = 0.0f;
    float m_Y = 0.0f;
};
```

Cette classe représente un gobelin qui patrouille, d'un bout à l'autre d'un couloir, dans un donjon, pour garder un trésor.



```
Goblin _goblin;
while (!_done) {
    for (float x = 0.0f; x < 100.0f; x++) {
        _goblin.SetX(x);
    }

    for (float x = 100.0f; x > 0.0f; x--) {
        _goblin.SetX(x);
    }

    Draw();
}
```

Le problème le plus apparent avec ce bout de code réside dans le fait que le personnage se déplace, aller-retour, sans jamais redonner le contrôle au système de rendu. Vous serez seulement en mesure d'apercevoir la patrouille à son **emplacement finale**! Vous devrez donc éliminer les boucles «for» et permettre au gobelin de bouger, petit à petit, à chaque mise à jour pour afficher chacune de ses positions, vous laissant ainsi observer la créature se mouvoir dans le jeu.

```
int main() {
    bool _done = false;
    Goblin _goblin;
    bool _movingLeft = true;
    float _x = 0.0f;

    while (!_done) {
        ProcessInput();

        if (_movingLeft == true) {
            _x--;

            if (_x == 0.0f) {
                _movingLeft = false;
            }
        }
        else {
            _x++;

            if (_x == 100.0f) {
                _movingLeft = true;
            }
        }

        _goblin.SetX(_x);

        Draw();
    }
}
```

Maintenant, vous pourrez voir le gobelin se déplacer, mais cette amélioration à un prix : le code à **légèrement gagner en complexité**.

Si vous voulez ajouter une autre patrouille, cette complexité ne cessera d'augmenter.

```
int main() {
    bool _done = false;

 Goblin _goblin;
    bool _movingLeft_goblin = true;
    float _x_goblin = 0.0f;
    Skeleton _skeleton;
    bool _movingLeft_skeleton = true;
    float _x_skeleton = 0.0f;
    while (!_done) {
        ProcessInput();

        if (_movingLeft_goblin == true) {
            _x_goblin--;
            if (_x_goblin == 0.0f) {
                _movingLeft_goblin = false;
            }
        }
        else {
            _x_goblin++;
            if (_x_goblin == 100.0f) {
                _movingLeft_goblin = true;
            }
        }

        if (_movingLeft_skeleton == true) {
            _x_skeleton--;
            if (_x_skeleton == 0.0f) {
                _movingLeft_skeleton = false;
            }
        }
        else {
            _x_skeleton++;
            if (_x_skeleton == 100.0f) {
                _movingLeft_skeleton = true;
            }
        }

        _goblin.SetX(_x_goblin);
        _skeleton.SetX(_x_skeleton);

        Draw();
    }
}
```

Et maintenant, vous avez envie de faire un jeu avec 100 monstres différents ? Ou d'autres types d'objets ? En fait, à chaque ajout, la difficulté augmente et ce n'est vraiment pas une situation souhaitable dans une grande production. Ce code se dirige tout droit vers la catastrophe. (*Notez ici que* ***vous ne devez pas vous trouver dans cette situation*** *pour votre remise finale*)

Pensez plutôt à chaque **«entités»** du jeu et ajoutez une méthode **«update»** qui encapsule la logique de celle-ci. À chaque mise à jour du jeu, vous appeler la méthode «update» pour laisser l'entité simuler elle-même une itération de sa propre logique. Nous pouvons simplement **séparer la logique (d'une entité) de la boucle** de jeu.

Le rôle de la boucle de jeu est de faire avancer le jeu dans le temps et le meilleur endroit pour simuler un gobelin est dans sa propre classe. Qui est le mieux placé pour obtenir les informations nécessaires à la simulation d'un gobelin, si ce n'est la classe «Goblin» elle-même ?

```
class Goblin {
public:
    void Update() {
        if (m_MovingLeft == true) {
            m_X--;

            if (m_X == 0.0f) {
                m_MovingLeft = false;
            }
        }
        else {
            m_X++;

            if (m_X == 100.0f) {
                m_MovingLeft = true;
            }
        }
    }

private:
    bool m_MovingLeft = false;
};
```

Dans la fonction principale, vous pouvez faire un ménage et grandement simplifier la lecture du code :

```
int main() {
    bool _done = false;
    Goblin _goblin;
    Skeleton _skeleton;

    while (!_done) {
        ProcessInput();

        _goblin.Update();
        _skeleton.Update();

        Draw();
    }
}
```

Nous avons réussi à contenir la complexité de notre boucle, mais elle ne fonctionne que pour des gobelins ou des squelettes. Avec une classe de base pour les **entités** et l'utilisation du **polymorphisme**, vous pourrez tenir une collection d'entités génériques qui peuvent être mises à jour tour à tour pour simuler leur logique de jeu de façon indépendante.

Entités

Les entités (*la classe de base des objets de notre jeu*) permettent de **généraliser** la séparation du code de jeu en objet logique. C'est l'équivalent des GameObject dans Unity.

Par exemple, dans le jeu «Pac-Man», on ne veut pas vraiment avoir le code des fantômes dans le même objet que celui qui contient les contrôles du joueur. Nous allons plutôt diviser le code en entité pour chaque fantôme et pour «pac-man». Cette séparation logique du code nous permet d'apporter des changements dans un, sans impact dans l'autre. Après tout, le code qui permet de contrôler la direction de «pac-man» ne devrait pas affecter le code qui déplace les fantômes.

Qu'on l'appelle «GameObject» ou «Actor», le concept reste le même : la séparation logique en «Entités». Pour implémenter de entités, nous allons ajouter une nouvelle classe et quelques méthodes :

```
class Entity
{
public:
    virtual ~Entity() = default;

    virtual void Start() {}
    virtual void Update(float dt) {}
    virtual void Draw() {}
    virtual void Destroy() {}
};
```

Question : *pourquoi utiliser des méthodes virtuelles si on n'implémente pas le corps de celle-ci ? Pourquoi ne pas utiliser des virtuelles pures ?*

La méthode «Update» permet d'actualiser la logique de l'entité qui pourra simuler un «frame». Chaque entité aura sa chance à chaque «frame». Dans vos jeux, une entité peut aussi rendre des graphiques. De la même façon que pour la logique, nous allons ajoutez une méthode «Draw» que le «jeu» appellera au bon moment (*normalement, le bon moment d'appeler «Draw» est entre le «Clear» et le «Present»*).

Le monde

Idéalement, nous avons besoin de regrouper toutes les entités à un endroit dans le jeu que nous appellerons **«le monde»**. Le monde a besoin d'une structure capable de contenir tous les entiers et d'itérer sur ceux-ci afin d'appeler les méthodes «Update» et «Draw». Par exemple, avec un «std::vector» comme dans l'exemple suivant :

```
class WorldService {
private:
    std::vector<Entity*> m_EntityInWorld;
};
```

Afin d'être en mesure de **rechercher**, **d'ajouter** ou **d'enlever** des entités à partir de n'importe où dans le code, le monde doit être contenu dans un **«Service»** et le **«Service Provider»** contient l'implémentation concrète de la mise à jour :

```
void Update(float dt) {
    for (auto entity : m_EntityInWorld) {
        entity->Update(dt);
    }
}

void Draw() {
    for (auto entity : m_EntityInWorld) {
        entity->Draw();
    }
}
```

Ajouter une entitée

Ajouter une entité au monde est simple : il suffit que de l'ajouter dans le tableau.

```
void Add(Entity* entity) {
    m_EntityInWorld.emplace_back(entity);
}
```

Et vous pouvez maintenant laisser le «monde» mettre à jour toutes les entités :

```
bool _done = false;
World _world;

_world.Add(new Goblin());
_world.Add(new Goblin());

while (!_done) {
    ProcessInput();

    _world.Update();
    _world.Draw();
}
```

Pour vous permettre d'utiliser la classe Goblin dans le «monde», elle doit premièrement hériter de «Entity».

```
class Goblin : public Entity {
public:
    virtual void Update();
    virtual void Draw();
    virtual void Destroy();

    void SetX(float x) { m_X = x; }
    void SetY(float y) { m_Y = y; }

private:
    float m_X = 0.0f;
    float m_Y = 0.0f;
};
```



Challenge 5.1 : *ajoutez un service de «world» dans l'engin et transférer tout le code «gameplay» dans une entité «test» (dans le projet «jeu») pour déresponsabiliser l'engin de ce qui devrait être côté jeu. Pour l'instant, vous pouvez utiliser qu'une Entitée monolithique, ce ne sont que nos tests.*

Trouver une entitée

Pour trouver une entité, vous pouvez vous servir du nom (ou l'id) de l'entité afin vous assurer que vous avez trouvé le bon objet avant de le retourner à l'utilisateur de la méthode.

```
class Entity {
public:
    Entity(const std::string& name) : m_Name(name) {}
    virtual ~Entity() = default;

    virtual void Start() {}
    virtual void Update(float dt) {}
    virtual void Draw() {}
    virtual void Destroy() {}

    std::string& GetName() { return m_Name; }

private:
    std::string m_Name;
};
```

La méthode la plus simple pour retrouver l'entité est d'itérer sur chaque élément du tableau jusqu'à ce que vous trouviez ce que vous cherchez. Dans le meilleur des cas, elle sera la première de la liste. Dans le pire des cas, elle ne sera même pas dans la liste et vous l'aurez parcouru pour rien :

```
Entity* Find(const std::string& name) {
    for (auto entity : m_EntityInWorld) {
        if (name == entity->GetName()) {
            return entity;
        }
    }

    return nullptr;
}
```

La boucle pour rechercher une entité demande par contre un certain temps et dépend de la quantité d'entités qu'elle contient (O(n)). Une façon plus rapide pour trouver une entité serait d'utiliser un **dictionnaire** en plus de notre tableau.

```
class World {
public:
    void Add(Entity* entity) {
        m_EntityInWorld.emplace_back(entity);
        m_EntityMap.emplace(entity->GetName(), entity);
    }

private:
    std::vector<Entity*> m_EntityInWorld;
    std::map<std::string, Entity*> m_EntityMap;
};
```

En utilisant le nom comme clé dans le dictionnaire (*ou une «map»*), vous pouvez retrouver (*ou savoir si elle n'est pas incluse*) l'entité directement par son nom (O(1)) :

```
Entity* Find(const std::string& name) {
    if (m_EntityMap.count(name) > 0) {
        return m_EntityMap[name];
    }

    return nullptr;
}
```

Note : *l'utilisation du «count» évite d'ajouter des «clée» pour rien dans la «map».*

Enlever une entitée

Pour supprimer une entité, vous devez l'enlever de toutes les structures que vous avez utilisées dans la classe «monde», et ne faire le «delete» qu'une seule fois sur celle-ci (*faire un delete sur un objet déjà détruit peut causer des problèmes en C++*).

```
void Remove(Entity* entity);
```

Pour un «std::vector», vous devez utiliser la méthode «erase» en fournissant un «itérateur». Il n'y a pas de méthode «remove».

Itérateurs

Un itérateur est un objet qui permet de parcourir une liste d'objets. L'organisation des données dans les structures fournies par «STD» n'est pas toujours la même. Par exemple, le «Vector» garde chaque donnée, une a la suite de l'autre, ce qui rend facile d'itérer les informations qu'elle contient en utilisant un index dans une boucle.

```
for (size_t i = 0; i < v.size(); i++)
{
    std::cout << v[i] << std::endl;
}
```

Mais, pour un dictionnaire, **les données n'y sont pas ajoutées séquentiellement** et itérer sur celui-ci, comme il n'utilise pas d'index, mais plutôt des clés, nécessite l'utilisation d'un itérateur.

```
std::vector<int> v = { 1, 3, 3, 4, 5, 6 };

for(auto it = v.begin(); it != v.end(); ++it) {

}
```

Manipulation de donnée dans une boucle

Dans le «monde» de votre jeu, vous avez besoin de méthodes pour ajouter, enlever et retrouver vos entités. Ces méthodes modifient le contenu de la liste d'entités pendant que le monde itère sur cette même liste. La modification d'une liste à l'intérieur d'une boucle est risquée et occasionne des erreurs parfois difficiles à comprendre. Voyons un exemple simple.

```
std::vector<int> v = { 1, 3, 3, 4, 5, 6 };

for(auto it = v.begin(); it != v.end(); ++it) {
    if (*it == 3) {
        v.erase(it);
    }
}
```

Lorsque vous effacez la valeur «3» de la liste, **l'itérateur n'est plus valide** et à la prochaine itération, le programme «crash».

La méthode «erase» retourne le prochain itérateur, il ne suffit alors que de mettre à jour notre variable «it» lorsqu'on détruit un élément de la liste :

```
std::vector<int> v = { 1, 3, 3, 4, 5, 6 };

for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it == 3) {
        it = v.erase(it);
    }
}
```

Par contre, ce code n'efface pas tous les «3», car notre itérateur passe par-dessus le deuxième «3» de la liste (*à cause du décalage lors de la suppression*). Il ne nous faut donc pas avancer l'itérateur lorsqu'on efface un item :

```
std::vector<int> v = { 1, 3, 3, 4, 5, 6 };

auto it = v.begin();
while(it != v.end()) {
    if (*it == 3) {
        it = v.erase(it);
    }
    else {
        ++it;
    }
}
```

Une autre solution serait de décrémenter l'itérateur lorsqu'on détruit un item :

```
std::vector<int> v = { 1, 3, 3, 4, 5, 6 };

for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it == 3) {
        v.erase(it--);
    }
}
```

Comme nous utilisons un «post-incrément», l'itérateur est reculé après avoir été utilisé par «erase».

Challenge 5.2 : *séparez le démo que nous avons en deux entités et, lorsqu'on appuie sur la barre d'espacement dans une entité, l'autre est détruite.*

Scènes

Avec toutes les entités regroupées dans un «monde», comment allons-nous faire pour ajouter le concept de «scene»? Nous avons besoin de savoir :

- quand nous pouvons créer des entités;
- quand les détruire.

On ne veut certainement pas modifier l'engin pour ajouter nos entités. Tout doit se faire dans le code du jeu. Pour ce qui est de la mise à jour et du rendu, le «World» s'en charge automatiquement sur chaque entité. La scène n'a pas besoin de le faire. Pour le «Load», ajoutez une interface que le jeu pourra implémenter pour chaque scène et le «monde» l'utilisera pour lancer le chargement de vos entités.

```
class IScene {
public:
    virtual ~IScene() = default;
    virtual void Load() = 0;
};
```

Pour supporter la gestion de scènes, vous devez ajouter une méthode pour enregistrer une implémentation concrète de l'interface et une méthode «load» pour appeler le chargement d'une scène par un nom.

```
void Load(const std::string& scene);
void Register(const std::string& name, IScene* scene);
```

«Register» vous permet de pré-instancier les scènes et de leur donner un nom. Ensuite avec «Load», nous pourrons changer de scène en utilisant ce nom. Le choix de la structure, pour contenir les scènes, est facile à faire: comme nous utilisons un nom pour les retrouver, ajoutez un «dictionnaire».

```
std::map<std::string, IScene*> m_Scenes;
```

Vous pouvez utiliser le dictionnaire pour retrouver une scène et appeler la méthode requise. Mais vous devez faire attention : si votre scène modifie les structures alors que vous itérez dessus, votre programme aura des erreurs.

```
void homer::World::Load(const std::string& scene) {
    if (m_Scenes.count(scene) > 0) {
        m_Scenes[scene]->Load();
    }
}

void homer::World::Register(const std::string& name, IScene* scene) {
    if (m_Scenes.count(name) == 0) {
        m_Scenes[name] = scene;
    }
}
```

Avant le «Load» d'une scène, il nous faut d'abord «UnLoad» la scène courante, s'il y en a une. Ajoutez une variable à «World», par exemple :

```
IScene* m_CurrentScene = nullptr;
```

Lorsqu'on change de scène, nous pourrons alors appeler une méthode «Destroy» sur chaque entité et vider la liste de «World». Par exemple :

```
void World::Unload() {
    if (m_CurrentScene != nullptr) {
        for (auto entity : m_EntityInWorld) {
            entity->Destroy();
            delete entity;
        }
        m_EntityInWorld.clear();
        m_EntityMap.clear();
    }
}

void World::Load(const std::string& scene) {
    if (m_Scenes.count(scene) > 0) {
        Unload();
        m_CurrentScene = m_Scenes[scene];
        m_CurrentScene->Load();
    }
}
```

Nous devons enregistrer les scènes avant de faire «start» de l'engin. Pour avoir le «world», nous devons initialiser le service. C'est pourquoi nous avons séparé l'initialisation du start pour nous laisser le temps de le faire.

Dans la méthode «load» de l'interface «IScene», vous pouvez créer les entités qui composent la scène. La logique du jeu se trouve, pour l'instant, dans la méthode «Update» de chaque entité.

Maintenant, pour faire un menu, vous devez créer une «IScene» et la charger dans l'engin. C'est cette classe qui dicte quelles entités sont disponibles dans une scène :

```
class MyDemo : public homer::IScene
{
public:
    virtual void Load() override;
};

void MyDemo::Load()
{
    homer::Entity* e1 = new TestEntity();
    homer::Entity* e2 = new DogEntity();

    homer::Engine::Get().World().Add(e1);
    homer::Engine::Get().World().Add(e2);
}
```

Challenge 5.3 : *séparez vos entités en deux scènes et lorsqu'on appuie sur la barre d'espacement, changez de scène.*